

00a — Preprocessing 2D

Dự án: Latent Manipulation of Brain MRI using Volume-Preserving GANs

Pipeline:

```
Raw .nii.gz
├─ skull-stripping (ANTsPyNet)
├─ resample 1mm → normalized/*.png
└─ giữ nguyên → unnormalized/*.png
```

Output:

```
MyDrive/preprocessed_2d/
├─ normalized/
├─ unnormalized/
└─ preprocessing_log.csv
```

Bước 1: Mount Google Drive + Cấu hình

```
from google.colab import drive
drive.mount('/content/drive')

import os

# ==== ĐƯỜNG DẪN ====
DATA_ROOT = '/content/drive/MyDrive/data'
TSV_PATH = '/content/drive/MyDrive/data/participants.tsv'
OUTPUT_ROOT = '/content/drive/MyDrive/preprocessed_2d'

# ==== SỐ SUBJECT XỬ LÝ ====
# Đổi thành 'ALL' để xử lý toàn bộ
NUM_SUBJECTS = 300

os.makedirs(f'{OUTPUT_ROOT}/normalized', exist_ok=True)
os.makedirs(f'{OUTPUT_ROOT}/unnormalized', exist_ok=True)

all_folders = sorted([f for f in os.listdir(DATA_ROOT) if
f.startswith('sub-BrainAge')])
print(f'Tổng folder : {len(all_folders)}')

Mounted at /content/drive
Tổng folder : 12654
```

Bước 2: Cài thư viện

```
!pip install antspyx antspynet nibabel -q

import ants
import antspynet
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')

print(f'ANTsPy version: {ants.__version__}')
print('Thư viện sẵn sàng!')
```

```
0.0/147.6 kB ? eta -:--:--
147.6/147.6 kB 6.4 MB/s eta
0:00:00
ents to build wheel ... etadata (pyproject.toml) ...
62.0/62.0 kB 7.1 MB/s eta
0:00:00
22.4/22.4 MB 62.0 MB/s eta
0:00:00
37.3/37.3 MB 24.9 MB/s eta
0:00:00
l) ...
```

Bước 3: Đọc participants.tsv

```
df = pd.read_csv(TSV_PATH, sep='\t', low_memory=False)

df_meta = df[['subject_id', 'subject_age', 'subject_sex',
              'subject_dx']].copy()
df_meta = df_meta.dropna(subset=['subject_id', 'subject_age'])
df_meta['subject_id'] = df_meta['subject_id'].astype(str).str.strip()
df_meta['subject_age'] = pd.to_numeric(df_meta['subject_age'],
                                       errors='coerce')
df_meta = df_meta.dropna(subset=['subject_age'])

age_dict = dict(zip(df_meta['subject_id'], df_meta['subject_age']))
sex_dict = dict(zip(df_meta['subject_id'], df_meta['subject_sex']))
dx_dict = dict(zip(df_meta['subject_id'], df_meta['subject_dx']))

folders_valid = sorted(list(set(all_folders) & set(age_dict.keys())))
folders = folders_valid if NUM_SUBJECTS == 'ALL' else
folders_valid[:NUM_SUBJECTS]

print(f'Subject dùng được : {len(folders_valid)}')
print(f'Sẽ xử lý : {len(folders)}')
```

```
print(f'Tuổi: min={df_meta["subject_age"].min():.1f},  
max={df_meta["subject_age"].max():.1f}')
```

Subject dùng được : 12654
Số xử lý : 300
Tuổi: min=5.0, max=97.0

Bước 4: Định nghĩa hàm preprocessing

```
def find_nii_file(subject_folder_path):  
    anat_path = os.path.join(subject_folder_path, 'anat')  
    if not os.path.exists(anat_path):  
        return None  
    for f in os.listdir(anat_path):  
        full_path = os.path.join(anat_path, f)  
        if os.path.isfile(full_path) and (f.endswith('.nii.gz') or  
f.endswith('.nii')):  
            return full_path  
        if os.path.isdir(full_path) and f.endswith('.nii'):  
            for inner_f in os.listdir(full_path):  
                inner_path = os.path.join(full_path, inner_f)  
                if os.path.isfile(inner_path) and  
(inner_f.endswith('.nii.gz') or inner_f.endswith('.nii')):  
                    return inner_path  
    return None  
  
def skull_strip(ants_img):  
    prob = antspynet.brain_extraction(ants_img, modality='t1')  
    mask = ants.threshold_image(prob, 0.5, 1.0, 1, 0)  
    return ants_img * mask  
  
def normalize_intensity(ants_img):  
    data = ants_img.numpy().astype(np.float32)  
    nonzero = data[data > 0]  
    if nonzero.size > 0:  
        p1, p99 = np.percentile(nonzero, [1, 99])  
        data = (np.clip(data, p1, p99) - p1) / (p99 - p1 + 1e-8)  
    return ants.from_numpy(data, origin=ants_img.origin,  
                           spacing=ants_img.spacing,  
                           direction=ants_img.direction)  
  
def resample_lmm(ants_img):  
    return ants.resample_image(ants_img, (1.0, 1.0, 1.0),  
                               use_voxels=False, interp_type=1)  
  
def save_axial_png(ants_img, output_path):
```

```
data = ants_img.numpy().astype(np.float32)
# Cắt axial slice giữa theo chiều Z (axis 2) – chuẩn hướng MRI
sl = data[:, :, data.shape[2]//2]
plt.imsave(output_path, sl.T, cmap='gray', origin='lower')
```

```
print('Hàm preprocessing định nghĩa xong!')
```

Hàm preprocessing định nghĩa xong!

Bước 5: Chạy preprocessing

Đổi `NUM_SUBJECTS = 'ALL'` ở Bước 1 để xử lý toàn bộ.

Nếu Colab bị ngắt, chạy lại sẽ **tự động bỏ qua** subject đã xử lý rồi.

□ **Log được ghi ngay sau mỗi subject** — không mất data dù sập giữa chừng.

```
import csv
from tqdm.notebook import tqdm

errors = []
skipped = 0
new_count = 0

# — Đọc log cũ để biết subject nào đã được ghi rồi
log_path = f'{OUTPUT_ROOT}/preprocessing_log.csv'
logged_ids = set()
if os.path.exists(log_path):
    logged_ids = set(pd.read_csv(log_path)['subject_id'].astype(str))
    print(f'Log cũ tìm thấy: {len(logged_ids)} subject đã có sẵn')
else:
    print('Chưa có log – sẽ tạo mới')

# — Chuẩn bị file CSV (append mode, ghi header nếu chưa có)
FIELDNAMES = ['subject_id', 'age', 'sex', 'dx', 'png_file']
write_header = not os.path.exists(log_path)
log_file = open(log_path, 'a', newline='', encoding='utf-8')
writer = csv.DictWriter(log_file, fieldnames=FIELDNAMES)
if write_header:
    writer.writeheader()

print(f'Dang xử lý {len(folders)} subjects...\n')

try:
    for subject_id in tqdm(folders, desc='Preprocessing 2D'):
        try:
            age = age_dict.get(subject_id, None)
            if age is None:
                errors.append({'subject_id': subject_id, 'error':
```

```

'Không có thông tin tuổi'})
    continue

    fname = f'{subject_id}_age{age:.1f}.png'

    # — Bo' qua nê'u PNG đã có VÀ đã được ghi vào log
    if (subject_id in logged_ids and
        os.path.exists(f'{OUTPUT_ROOT}/normalized/{fname}'))
and
os.path.exists(f'{OUTPUT_ROOT}/unnormalized/{fname}')):
    skipped += 1
    continue

    nii_path = find_nii_file(os.path.join(DATA_ROOT,
subject_id))
    if nii_path is None:
        errors.append({'subject_id': subject_id, 'error':
'Không tìm thấy .nii'})
        continue

    img = ants.image_read(nii_path)
    brain = skull_strip(img)

    # UNNORMALIZED
    save_axial_png(normalize_intensity(brain),
f'{OUTPUT_ROOT}/unnormalized/{fname}')
    # NORMALIZED
    save_axial_png(normalize_intensity(resample_1mm(brain)),
f'{OUTPUT_ROOT}/normalized/{fname}')

    # — Ghi ngay vào CSV sau khi xử lý xong 1 subject
    row = {
        'subject_id': subject_id,
        'age' : age,
        'sex' : sex_dict.get(subject_id, 'unknown'),
        'dx' : dx_dict.get(subject_id, 'unknown'),
        'png_file' : fname
    }
    writer.writerow(row)
    log_file.flush() # ghi xuông disk ngay, không giữ trong
buffer

    new_count += 1

except Exception as e:
    errors.append({'subject_id': subject_id, 'error': str(e)})

finally:

```

```
log_file.close() # đảm bảo file luôn được đóng dù có lỗi hay  
bị ngắt
```

```
print(f'\n=== HOÀN THÀNH ===')  
print(f'Xử lý mới : {new_count}')
```

print(f'Bỏ qua : {skipped} (đã xử lý trước đó)')

```
print(f'Lỗi : {len(errors)}')
```

print(f'Log tại : {log_path}')

```
if errors:  
    for e in errors:  
        print(f" - {e['subject_id']}: {e['error']}")
```

Đang xử lý 300 subjects...

```
{"model_id":"cfd1178da764427ae96b334eb265787","version_major":2,"version_minor":0}
```

Downloading data from <https://ndownloader.figshare.com/files/348218745683832/5683832> _____ 1s 0us/step

Downloading data from <https://ndownloader.figshare.com/files/2259717514969865/14969865> _____ 1s 0us/step

WARNING:tensorflow:5 out of the last 5 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x7d2a83434680> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

WARNING:tensorflow:6 out of the last 6 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x7d2a80b1bec0> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

```
=== HOÀN THÀNH ===  
Xử lý mới : 299
```

```
Bỏ qua      : 0 (đã xử lý trước đó)
Lỗi         : 1
Output tại  : /content/drive/MyDrive/preprocessed_2d
             - sub-BrainAge000000:
/project/itksource/Modules/Filtering/ImageStatistics/include/itkImageM
omentsCalculator.hxx:123:
ITK ERROR: ImageMomentsCalculator(0x34acc380): Compute(): Total Mass
of the image was zero. Aborting here to prevent division by zero later
on.
```